L11: MySQL Functions

Sirawich.V

Ref. https://www.mysqltutorial.org/

RECAP from Lab #10

- PROCEDURE

```
DELIMITER $$
CREATE PROCEDURE GetTotalOrder()
BEGIN
	DECLARE totalOrder INT DEFAULT 0;
	SELECT
		COUNT(*)
	INTO totalOrder FROM
		orders;
	SELECT totalOrder;
END$$
DELIMITER ;
```

- PROCEDURE

```
DELIMITER $$
CREATE FUNCTION CustomerLevel(
	credit DECIMAL(10,2)
)
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
    DECLARE customerLevel VARCHAR(20);
    IF credit > 50000 THEN
                SET customerLevel = 'PLATINUM';
    ELSEIF (credit >= 50000 AND credit <=
10000) THEN
        SET customerLevel = 'GOLD';
    ELSEIF credit < 10000 THEN
        SET customerLevel = 'SILVER';
    END IF;
      RETURN (customerLevel);
END$$
DELIMITER ;
```

11.1 Conditional Statements

IF statement

The IF statement allows you to evaluate one or more conditions and execute the corresponding code block if the condition is true.

Syntax

```
IF condition THEN
   statements;
ELSEIF elseif-condition THEN
   statements;
...
ELSE
   statements;
END IF;
```

Conditional Statements (IF)

Example

```
DELIMITER $$

CREATE PROCEDURE GetCustomerLevel(
    IN  pCustomerNumber INT,
    OUT pCustomerLevel  VARCHAR(20))
BEGIN
    DECLARE credit DECIMAL DEFAULT 0;

    SELECT creditLimit INTO credit FROM customers
    WHERE customerNumber = pCustomerNumber;

    IF credit > 50000 THEN
        SET pCustomerLevel = 'PLATINUM';
    ELSEIF credit <= 50000 AND credit > 10000 THEN
        SET pCustomerLevel = 'GOLD';
    ELSE
        SET pCustomerLevel = 'SILVER';
    END IF;
END $$
DELIMITER ;
```

customerNu	customerName	contactLastName	creditLimit
424	Classic Legends Inc.	Hernandez	67500.00
447	Gift Ideas Corp.	Lewis	49700.00
448	Scandinavian Gift Ideas	Larsson	116400.00
450	The Sharp Gifts Warehouse	Frick	77600.00
NULL	NULL	NULL	NULL

Call

```
CALL GetCustomerLevel(447, @level);
SELECT @level;
```

@level
GOLD

Conditional Statements

CASE statement has two forms:

- Simple CASE statement
- Searched CASE statement (equivalent to the IF statement)

Syntax: Simple

```
CASE case_value
    WHEN when_value1 THEN
        statements ;
    WHEN when_value2 THEN
        statements ;
    ...
    [ELSE else-statements]
END CASE;
```

Syntax: Searched

```
CASE
    WHEN search_condition1 THEN
        statements ;
    WHEN search_condition1 THEN
        statements ;
    ...
    [ELSE else-statements]
END CASE;
```

Conditional Statements (CASE)

Example

```
DELIMITER $$

CREATE PROCEDURE GetCustomerShipping(
  IN pCustomerNumber INT,
  OUT pShipping VARCHAR(50)
)
BEGIN
        DECLARE customerCountry VARCHAR(100);
        SELECT country INTO customerCountry
        FROM customers
        WHERE
          customerNumber = pCustomerNumber;

        CASE customerCountry
                WHEN 'USA' THEN
                        SET pShipping = '2-day Shipping';
                WHEN 'Canada' THEN
                        SET pShipping = '3-day Shipping';
                ELSE
                        SET pShipping = '5-day Shipping';
        END CASE;
END$$
DELIMITER ;
```

customerNu	customerName	country	contactLastN
103	Atelier graphique	France	Schmitt
112	Signal Gift Stores	USA	King
114	Australian Collectors, Co.	Australia	Ferguson
119	La Rochelle Gifts	France	Labrune

Call

```
CALL GetCustomerShipping(112,
                         @shipping);
SELECT @shipping;
```

@shipping
2-day Shipping

Conditional Statements (CASE)

Example: Searched

```
CASE
    WHEN waitingDay < 0 THEN
            SET pDeliveryStatus = 'Early Delivery';

    WHEN waitingDay = 0 THEN
            SET pDeliveryStatus = 'On Time';

    WHEN waitingDay >= 1 AND waitingDay < 5 THEN
            SET pDeliveryStatus = 'Late';

    WHEN waitingDay >= 5 THEN
            SET pDeliveryStatus = 'Very Late';

    ELSE
            SET pDeliveryStatus = 'No Information';
END CASE;
```

If the ELSE clause is not available and the CASE cannot find any matches, it'll issue the following error:

```
Case not found for CASE statement
```

11.2 Loops

The LOOP statement allows you to execute one or more statements repeatedly.

Typically, you terminate the loop when a condition is true by using IF and LEAVE statements.

Syntax

```
[label]: LOOP
     ...

END LOOP;
```

Syntax: With loop breaker

```
[label]: LOOP
     ...

    -- terminate the loop
    IF condition THEN
        LEAVE [label];
    END IF;
END LOOP;
```

Loops

Example:

```
DELIMITER //
CREATE PROCEDURE fillDates(
     IN startDate DATE,
     IN endDate DATE
)
BEGIN
     DECLARE currentDate DATE DEFAULT startDate;
     insert_date: LOOP

          INSERT INTO calendars(date, month, quarter, year)
          VALUES(currentDate, MONTH(currentDate), QUARTER(currentDate), YEAR(currentDate));

          SET currentDate = DATE_ADD(currentDate, INTERVAL 1 DAY);
          -- Break Loop
          IF currentDate > endDate THEN
                    LEAVE insert_date;
          END IF;

     END LOOP;
END //
DELIMITER ;
```

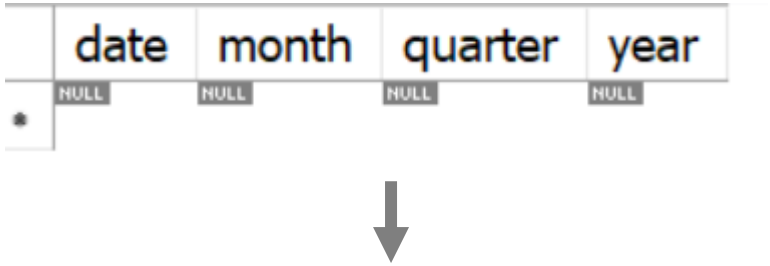
Loops

Example:

```
CREATE TABLE calendars (
    date DATE PRIMARY KEY,
    month INT NOT NULL,
    quarter INT NOT NULL,
    year INT NOT NULL
);

CALL fillDates('2024-01-01','2024-12-31');

SELECT * FROM calendars;
```



date	month	quarter	year
2024-02-02	2	1	2024
2024-02-03	2	1	2024
2024-02-04	2	1	2024
2024-02-05	2	1	2024

11.3 While Loops & Repeat Loops

The WHILE loop is a loop statement that executes a block of code repeatedly as long as a condition is true.

Syntax

```
[begin_label:] WHILE search_condition DO
    statement_list;
END WHILE [end_label]
```

Do when True.

The REPEAT statement creates a loop that repeatedly executes a block of statements until a condition is true.

Syntax

```
[begin_label:] REPEAT
    statemen;
UNTIL condition
END REPEAT [end_label]
```

Do when False.

While Loops

Example:

```
DELIMITER //
CREATE PROCEDURE fillDates(
     IN startDate DATE,
     IN endDate DATE
)
BEGIN
     DECLARE currentDate DATE DEFAULT startDate;

     insert_date: WHILE currentDate <= endDate DO
          INSERT INTO calendars(date, month, quarter, year)
          VALUES(currentDate, MONTH(currentDate), QUARTER(currentDate), YEAR(currentDate));

          SET currentDate = DATE_ADD(currentDate, INTERVAL 1 DAY);
     END WHILE;

END //
DELIMITER ;
```

Repeat Loops

Example:

```
DELIMITER //
CREATE PROCEDURE fillDates(
     IN startDate DATE,
     IN endDate DATE
)
BEGIN
     DECLARE currentDate DATE DEFAULT startDate;

     insert_date: REPEAT
          INSERT INTO calendars(date, month, quarter, year)
          VALUES(currentDate, MONTH(currentDate), QUARTER(currentDate), YEAR(currentDate));
          SET currentDate = DATE_ADD(currentDate, INTERVAL 1 DAY);
     UNTIL currentDate > endDate
     END REPEAT;

END //
DELIMITER ;
```

Repeat Loop is equivalent to Normal loop with Breaker.